



HACKATHON PROJECT COMPENDIUM

Digital Twins · Industry 5.0 · 3D Visualisation · AR/VR

Mechanical & Construction Industry Focus | Trimiti Innovations Pvt. Ltd., Pune

ID	Project Title	Domain
P-01	AI-Powered Scan-to-BIM Digital Twin for Construction Progress Monitoring	Digital Twins Construction
P-02	AR Work-Instruction Overlay System for Shop-Floor Assembly in SME Manufacturing	AR/VR Industry 5.0 Manufacturing
P-03	Digital Twin-Driven Predictive Maintenance Dashboard for CNC Machine Tools	Digital Twins Industry 5.0 Manufacturing
P-04	VR-Based Immersive Safety Training Simulator for Construction Site Hazards	AR/VR Construction Industry 5.0
P-05	Agentic AI Co-Pilot for Structural Engineering Design Review and Code Compliance Check	Digital Twins 3D Visualisation AI Construction
P-06	Digital Twin for Port Logistics — Simulating Inbound/Outbound Flow and Storage Monitoring	Digital Twins Logistics 3D Visualisation
P-07	AI-Powered 2D Wire Harness Diagram Scanner with Automatic Component Identification	AI/ML 3D Visualisation Manufacturing Electronics
P-08	MCP Server for Eyeshot CAD — Enabling AI Agents to Drive 3D CAD Operations via Natural Language	AI Agents 3D Visualisation CAD Industry 5.0

AI-Powered Scan-to-BIM Digital Twin for Construction Progress Monitoring

1. PROBLEM STATEMENT

Construction projects lack real-time, automated mechanisms to compare as-built site conditions against design intent, resulting in costly rework and schedule overruns.

2. PROBLEM DESCRIPTION

On large-scale construction and infrastructure projects, tracking physical progress against BIM design models requires manual site inspections, which are infrequent, error-prone, and slow. Deviations from design — misaligned structural elements, incorrect MEP routing, or missing components — are often discovered late when correction is expensive. The absence of an automated scan-to-model comparison loop means project managers rely on subjective site reports rather than objective geometric evidence. This gap is especially acute in India's rapidly expanding infrastructure sector, where speed, compliance with RERA timelines, and quality assurance are paramount.

3. EXPECTED SOLUTION

Build a prototype that ingests point cloud data (from LiDAR or photogrammetry) and automatically aligns it against a reference IFC/BIM model. The system should highlight geometric deviations above a configurable threshold and generate a construction progress report with percentage completion per floor/zone. A lightweight web dashboard should visualise the 3D deviation heatmap overlaid on the BIM model. Bonus: integrate a conversational AI interface to query site status in natural language (e.g., "What is the slab completion on Level 3?").

4. RECOMMENDED TECHNOLOGY STACK

- Point Cloud Processing: Open3D, PDAL, or CloudCompare CLI for registration and deviation analysis
- BIM/IFC Parsing: IfcOpenShell (Python) to extract geometric representations from IFC files
- 3D Visualisation: Three.js or Potree for browser-based point cloud and BIM overlay rendering
- AI/ML: Segment Anything Model (SAM) or PointNet++ for automated object recognition in point clouds
- Backend / API: FastAPI (Python) with PostgreSQL; frontend in React; deployment on Docker
- Conversational AI: LangChain + OpenAI/Claude API for natural language progress query interface

5. REFERENCES

1. IfcOpenShell – Open source IFC library: <https://ifcopenshell.org>
2. Open3D – Point cloud processing: <http://www.open3d.org>
3. Potree – WebGL point cloud viewer: <https://potree.org>
4. Chen et al. (2022), 'Automated progress monitoring using point cloud and BIM', Automation in Construction, Vol. 140
5. BuildingSMART IFC Standard: <https://www.buildingsmart.org/standards/bsi-standards/industry-foundation-classes/>

AR Work-Instruction Overlay System for Shop-Floor Assembly in SME Manufacturing

1. PROBLEM STATEMENT

Small and medium manufacturing enterprises lack affordable, hands-free augmented reality tools to deliver real-time, step-by-step assembly guidance on the shop floor, leading to quality defects and high operator training costs.

2. PROBLEM DESCRIPTION

Industry 5.0 envisions seamless human-machine collaboration, yet most Indian SMEs still rely on paper-based assembly instructions or static PDF work orders displayed on shared tablets. This approach fails to adapt to the operator's viewpoint, does not detect assembly errors in real time, and makes training new workers slow and inconsistent. AR-based work instructions, proven in Tier-1 automotive and aerospace, remain out of reach for SMEs due to high hardware costs and complex deployment. A lightweight, web-based AR prototype using commodity smartphones or low-cost headsets could democratise this capability and directly support Make in India quality goals.

3. EXPECTED SOLUTION

Develop a WebAR prototype (browser-based, no app install) that recognises a printed fiducial marker or CAD model silhouette on a physical assembly jig and overlays animated, step-by-step 3D assembly instructions. The system should track operator step completion (tap-to-advance or voice command) and flag a warning when a part is placed in the wrong orientation (using device camera + CV). A QR-code-based content management portal should allow supervisors to author and update instruction sets without coding. Bonus: add a digital twin synchronisation trigger that logs completed steps to a cloud dashboard.

4. RECOMMENDED TECHNOLOGY STACK

- WebAR: AR.js + A-Frame (marker-based) or 8th Wall for image-target tracking — runs in mobile browser
- 3D Content: Three.js / Babylon.js for animated GLTF/GLB assembly step models
- Computer Vision: TensorFlow.js (MobileNet or custom classifier) for in-browser part orientation detection
- CMS / Backend: Supabase (PostgreSQL + Auth + Storage) for instruction set management; REST API
- Digital Twin Sync: MQTT broker (Mosquitto) + Node-RED for event streaming to cloud dashboard
- Hardware: Standard Android/iOS smartphone; optional Meta Quest 3 browser mode for immersive view

5. REFERENCES

1. AR.js – Web-based Augmented Reality: <https://ar-js-org.github.io/AR.js-Docs/>
2. A-Frame WebXR framework: <https://aframe.io>
3. Industry 5.0 – European Commission Vision: <https://research-and-innovation.ec.europa.eu>
4. Webel & Bockholt (2023), 'AR for Industrial Assembly Guidance', IEEE ISMAR Proceedings
5. TensorFlow.js Model Zoo: <https://www.tensorflow.org/js/models>

Digital Twin–Driven Predictive Maintenance Dashboard for CNC Machine Tools

1. PROBLEM STATEMENT

Unplanned downtime of CNC machine tools in discrete manufacturing causes significant production losses, yet most SMEs lack affordable real-time condition monitoring and predictive maintenance capability.

2. PROBLEM DESCRIPTION

CNC machine tools are the heartbeat of precision manufacturing, but their health is typically assessed by periodic manual inspection or rudimentary threshold alarms on PLCs. Vibration signatures, spindle current anomalies, thermal drift, and tool wear patterns — all detectable through sensors — are rarely correlated into a unified predictive model. The consequence is unexpected failures mid-production, scrap parts, and missed delivery commitments. A Digital Twin that mirrors the live machine state and continuously evaluates failure probability using ML models could shift maintenance from reactive to predictive, reducing unplanned downtime by 20–40% as demonstrated in benchmark studies.

3. EXPECTED SOLUTION

Build a Digital Twin prototype for a simulated or real CNC lathe/milling machine. Ingest synthetic or real sensor streams (vibration, temperature, spindle load, tool-life counter) via OPC-UA or MQTT. Train a lightweight anomaly detection model (Isolation Forest or LSTM-Autoencoder) on historical run data. Visualise the live machine state in a 3D GLTF model in the browser, colour-coding subsystems by health score. Generate automated maintenance alerts with root-cause suggestions. Bonus: integrate a FMEA knowledge graph to map detected anomalies to failure modes and recommended corrective actions.

4. RECOMMENDED TECHNOLOGY STACK

- Connectivity: OPC-UA (node-opcua library) or MQTT (Mosquitto) for real-time sensor data ingestion
- ML / Anomaly Detection: Python scikit-learn (Isolation Forest), PyTorch (LSTM-Autoencoder), or Prophet for time-series
- Digital Twin Engine: Eclipse Ditto (open-source) or custom REST API mirroring machine state as JSON Thing model
- 3D Visualisation: Babylon.js with GLTF machine model; colour-map overlay driven by health score API
- Dashboard: Grafana (time-series charts) + React-based 3D viewer; InfluxDB for sensor data storage
- Knowledge Graph (Bonus): Neo4j + LangChain for FMEA-linked root-cause reasoning via NL query

5. REFERENCES

1. Eclipse Ditto – Open-source Digital Twin framework: <https://eclipse.dev/ditto/>
2. OPC Foundation UA Standard: <https://opcfoundation.org/about/opc-technologies/opc-ua/>
3. Zhao et al. (2023), 'LSTM-based predictive maintenance for CNC machining', Journal of Manufacturing Systems, Vol. 66
4. ISO 13374 – Condition monitoring and diagnostics of machines: <https://www.iso.org/standard/78525.html>
5. Grafana + InfluxDB IoT stack: <https://grafana.com/docs/grafana/latest/getting-started/>

VR-Based Immersive Safety Training Simulator for Construction Site Hazards

1. PROBLEM STATEMENT

Construction remains one of the most accident-prone industries globally, and traditional classroom-based safety training fails to build intuitive hazard-recognition reflexes that workers need on dynamic job sites.

2. PROBLEM DESCRIPTION

India's construction sector accounts for a disproportionate share of occupational fatalities, with falls-from-height, electrocution, and struck-by incidents being the leading causes. Conventional safety training relies on videos, posters, and toolbox talks — formats that engage only declarative memory and do not simulate the stress or spatial complexity of real hazard scenarios. Research consistently shows that immersive VR training significantly improves hazard recognition recall and behavioural compliance compared to passive methods. Building an accessible, low-cost VR training module — deployable on standalone headsets or even Google Cardboard with a smartphone — would make immersive safety training viable for India's fragmented construction workforce.

3. EXPECTED SOLUTION

Develop a VR scenario in which the trainee navigates a 3D construction site (multi-storey RC frame under construction) and must identify, tag, and remediate at least five hazard types: unprotected edge, missing PPE, improper scaffolding, live electrical panel, and chemical storage violation. Each identified hazard triggers an explanatory animation with the applicable IS/BIS safety code reference. A scoring dashboard tracks performance across trainees and sessions. Bonus: add a multiplayer mode where a trainer can remotely observe and annotate the VR scene in real time.

4. RECOMMENDED TECHNOLOGY STACK

- VR Engine: Unity 3D (XR Interaction Toolkit) or Unreal Engine 5 with OpenXR support
- Target Hardware: Meta Quest 2/3 (standalone) or WebXR for browser-based fallback on desktop
- 3D Assets: Sketchfab / Quixel Megascans for construction site props; custom RC frame in Blender
- Scoring & Analytics: Firebase Realtime Database for session score tracking; React dashboard for site managers
- Multiplayer (Bonus): Photon PUN2 or Unity Netcode for GameObjects for real-time co-presence
- Regulatory Content: IS 3696 (Safety in Scaffolding), IS 5216 (Electrical Safety), BIS portal references

5. REFERENCES

1. Sacks et al. (2013), 'Virtual construction simulation and training', Automation in Construction, Vol. 35
2. Unity XR Interaction Toolkit docs: <https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.0/manual/>
3. Meta Quest Developer Hub: <https://developer.oculus.com>
4. BIS Safety Standards Portal: <https://www.bis.gov.in/>
5. Lucas et al. (2021), 'Effectiveness of VR safety training', Safety Science, Vol. 134

Agentic AI Co-Pilot for Structural Engineering Design Review and Code Compliance Check

1. PROBLEM STATEMENT

Structural engineers spend a disproportionate amount of time manually cross-checking design drawings against multiple IS/BIS codes, a repetitive cognitive task that is error-prone and delays project approvals.

2. PROBLEM DESCRIPTION

Structural design review in India requires simultaneous compliance with IS 456 (RCC), IS 800 (Steel), IS 1893 (Seismic), and NBC 2016, among others. Engineers manually parse hundreds of pages of code provisions for each design iteration, a process that is slow, inconsistent, and a bottleneck in project delivery. Mistakes that escape review can result in structural deficiencies caught only at the construction stage — or worse, after occupancy. An AI agent that understands both the geometric intent of a structural model (via IFC parsing) and the regulatory text (via RAG over code documents) could autonomously flag non-compliances, suggest code-compliant alternatives, and dramatically compress review cycles.

3. EXPECTED SOLUTION

Build an agentic AI prototype that accepts an IFC structural model as input and a set of IS code PDFs as a knowledge base. The agent should parse member geometry, material tags, and load case annotations from the IFC file, then use a RAG pipeline to query the code documents for relevant clauses. It should output a structured compliance report listing each checked element, the applicable clause, the design value, the code limit, and pass/fail status. A 3D viewer should colour-code non-compliant members in the model. Bonus: add an agentic loop where the system proposes a revised section size for failed members and re-checks compliance automatically.

4. RECOMMENDED TECHNOLOGY STACK

- IFC Parsing: IfcOpenShell (Python) to extract structural member geometry, material, and properties
- RAG Pipeline: LangChain + FAISS or Chroma vector store; PDF ingestion via PDFPlumber or Unstructured.io
- LLM: Claude 3.5 Sonnet / GPT-4o via API for clause interpretation and compliance reasoning
- 3D Viewer: xeokit-sdk (open-source BIM viewer) for IFC rendering with colour-coded compliance overlay
- Agentic Loop (Bonus): LangGraph or CrewAI for multi-step agent that re-checks after proposing fixes
- Report Generation: Python-docx or ReportLab for structured compliance report output in DOCX/PDF

5. REFERENCES

1. IfcOpenShell Python API: <https://ifcopenshell.org/docs/>
2. xeokit BIM Viewer SDK: <https://xeokit.io>
3. LangChain RAG documentation: https://python.langchain.com/docs/use_cases/question_answering/
4. IS 456:2000 – Plain and Reinforced Concrete Code, BIS: <https://www.bis.gov.in>
5. Anthropic Claude API: <https://docs.anthropic.com>

Digital Twin for Port Logistics — Simulating Inbound/Outbound Flow and Storage Monitoring

1. PROBLEM STATEMENT

Seaports and inland container depots struggle with dynamic congestion, suboptimal berth and yard utilisation, and opaque cargo dwell times due to the absence of a real-time, simulation-capable Digital Twin of their inbound and outbound logistics operations.

2. PROBLEM DESCRIPTION

Modern ports handle thousands of container moves per day across berths, quay cranes, yard tractors, stacking areas, and gate lanes. Without a unified real-time model, planners rely on spreadsheets and siloed TOS (Terminal Operating System) dashboards that cannot simulate 'what-if' scenarios — such as the impact of a vessel delay cascading into yard congestion. Inventory accuracy in open-yard storage degrades rapidly as containers are shifted without GPS-level tracking. Port Authorities under India's Sagarmala Programme and JNPT/Adani Port expansions urgently need data-driven simulation tools to increase throughput and reduce turnaround time, directly impacting export competitiveness.

3. EXPECTED SOLUTION

Build a Digital Twin prototype that ingests real-time or simulated AIS (Automatic Identification System) vessel data and TOS event streams, and maintains a live 3D model of the port yard, berths, and gate lanes. The system should run a discrete-event simulation (DES) engine alongside the live twin to model inbound/outbound container flows and compute KPIs: berth occupancy, yard utilisation, average dwell time, and gate queue length. A 3D browser-based dashboard should visualise the port layout with colour-coded zone alerts. Bonus: implement an AI scheduling agent that recommends crane/truck allocation to minimise vessel turnaround time.

4. RECOMMENDED TECHNOLOGY STACK

- Digital Twin Platform: Eclipse Ditto or custom REST/MQTT-based thing model; AIS data via AISStream.io API
- Discrete-Event Simulation: SimPy (Python) for container flow modelling; or AnyLogic (free academic) for agent-based port DES
- 3D Visualisation: CesiumJS (geospatial 3D) or Three.js with OSM-derived port layout; GLTF crane/vessel models from Sketchfab
- Data Ingestion: Apache Kafka for high-throughput event streaming from TOS/AIS; TimescaleDB for time-series storage
- AI Scheduling Agent (Bonus): LangGraph + OR-Tools (Google) for constraint-based berth and crane allocation recommendations
- Standards: UN/CEFACT trade document standards; IMO AIS NMEA 0183 message format; Sagarmala Programme digital port guidelines

5. REFERENCES

1. AISStream.io – Real-time AIS vessel tracking API: <https://aisstream.io>
2. SimPy – Discrete-event simulation in Python: <https://simpy.readthedocs.io>
3. CesiumJS – Geospatial 3D engine: <https://cesium.com/platform/cesiumjs/>
4. Sagarmala Programme – MoPSW Digital Ports Initiative: <https://sagarmala.gov.in>
5. Vis et al. (2003), 'Transshipment of containers at a container terminal: An overview', European Journal of Operational Research, Vol. 147(1)
6. Eclipse Ditto – Open-source Digital Twin framework: <https://eclipse.dev/ditto/>

AI-Powered 2D Wire Harness Diagram Scanner with Automatic Component Identification

1. PROBLEM STATEMENT

Engineering teams in automotive and industrial electronics waste significant man-hours manually digitising legacy 2D wire harness drawings to extract netlist data, connector inventories, and splice locations, with frequent transcription errors causing costly rework.

2. PROBLEM DESCRIPTION

Wire harness drawings — the schematic backbone of automotive, aerospace, and industrial machinery wiring — are often stored as scanned PDFs or TIFF images from legacy projects, with no structured digital representation. Modern harness design tools such as Splice CAD and RapidHarness require data in structured formats (netlist, BOM, connector pin-out) that must currently be entered manually from these drawings. The component vocabulary is rich and domain-specific: splice points, connectors, terminals, cables, shielding symbols, and ground references each carry distinct visual signatures that standard OCR cannot decode. An AI system that can parse these drawings into machine-readable harness data would dramatically accelerate re-engineering, ECO (Engineering Change Order) management, and migration to digital-first harness design workflows.

3. EXPECTED SOLUTION

Build a prototype that accepts a scanned or photographed 2D wire harness schematic image and automatically identifies and labels: connectors (with pin count and gender), wire segments (colour-coded by net), splice points, ground symbols, and shield terminations. The output should be a structured JSON netlist and a BOM table exportable to CSV. A web interface should overlay detected components as interactive annotations on the original image, allowing engineers to correct misidentifications. Bonus: auto-generate a Splice CAD-compatible harness file or a RapidHarness-importable XML from the extracted netlist, enabling one-click transition to a digital harness design environment.

4. RECOMMENDED TECHNOLOGY STACK

- Computer Vision / Object Detection: YOLOv8 or RT-DETR (Ultralytics) fine-tuned on a custom harness symbol dataset for component detection
- OCR & Symbol Recognition: PaddleOCR for wire labels and net names; custom CNN classifier for harness symbol types (splice, ground, shield)
- VLM for Context: GPT-4o Vision or Claude 3.5 Sonnet (vision) for high-level drawing interpretation and ambiguous symbol resolution
- Backend: FastAPI (Python) + OpenCV for image pre-processing (deskew, binarise, segment); PostgreSQL for netlist storage
- Frontend: React + Fabric.js for interactive annotation overlay; export to JSON, CSV, and Splice CAD splice-py format
- Reference Tools: Splice CAD Python library (splice-py on GitHub) for output validation; RapidHarness XML schema for BOM export

5. REFERENCES

1. Splice CAD – Web-based cable and harness design tool: <https://splice-cad.com>
2. Splice-py – Python library for programmatic harness creation: <https://github.com/splice-cad/splice-py>
3. RapidHarness – Full lifecycle wire harness CAD software: <https://rapidharness.com>
4. Ultralytics YOLOv8 documentation: <https://docs.ultralytics.com>
5. PaddleOCR – Multilingual OCR toolkit: <https://github.com/PaddlePaddle/PaddleOCR>
6. IPC-620 – Requirements and Acceptance for Cable and Wire Harness Assemblies (IPC Standard)

MCP Server for Eshot CAD — Enabling AI Agents to Drive 3D CAD Operations via Natural Language

1. PROBLEM STATEMENT

Engineers using Eshot-based .NET CAD applications cannot leverage AI agents to automate 3D modelling, drawing generation, or geometry analysis tasks because no standardised AI-to-CAD communication protocol exists for the Eshot SDK.

2. PROBLEM DESCRIPTION

Eshot by devDept is a powerful .NET CAD SDK supporting BRep modelling, drafting, point cloud processing, FEM simulation, and multi-format translation (STEP, IGES, IFC, DWG, glTF, E57). Despite its rich programmatic API, there is currently no bridge that allows Large Language Model (LLM) agents to orchestrate Eshot operations — such as loading a STEP file, querying geometry properties, generating section views, or exporting to a target format — through natural language commands. The Model Context Protocol (MCP), introduced by Anthropic, provides a standardised interface for AI agents to invoke tools and resources. An MCP server wrapping Eshot's .NET API would make every Eshot operation accessible to Claude, GPT-4o, or any MCP-compatible agent, enabling a new class of conversational CAD workflows.

3. EXPECTED SOLUTION

Build an MCP server (using the MCP .NET SDK or a Python bridge via Eshot's COM/REST interface) that exposes a curated set of Eshot operations as MCP tools. Minimum viable toolset: `load_model` (STEP/IGES/IFC/OBJ), `list_entities` (returns entity types, IDs, bounding boxes), `get_entity_properties` (mass, surface area, volume), `create_section_view` (plane definition, output SVG), `export_model` (target format selection), and `measure_distance` (between two entity IDs). The MCP server should connect to a Claude or GPT-4o agent session, enabling prompts like 'Load the turbine_blade.STEP file, compute its volume, and export a top-view drawing as PDF.' Bonus: add a `change_material` tool that modifies visual properties and regenerates a rendered screenshot returned as a base64 image resource.

4. RECOMMENDED TECHNOLOGY STACK

- MCP Server Runtime: Anthropic MCP .NET SDK (C#) or Python MCP SDK (mcp PyPI package) with JSON-RPC 2.0 transport over stdio or SSE
- Eshot Integration: devDept Eshot Pro/Ulimate .NET SDK (C#); WinForms/WPF host process exposing operations via named pipe or local REST API
- Language Bridge (if Python MCP): Python.NET (pythonnet) or a lightweight ASP.NET Core REST wrapper around Eshot for cross-language calls
- Agent Runtime: Claude claude-sonnet-4-6 via Anthropic API with MCP tool_use capability; or LangChain MCP adapter
- Supported Formats: STEP, IGES, IFC, DWG/DXF, glTF, OBJ, STL, E57 (point cloud) — all natively supported by Eshot translators
- Testing & Validation: MCP Inspector (Anthropic CLI tool) for tool schema validation; pytest for Python bridge integration tests

5. REFERENCES

1. Eshot Pro – .NET CAD SDK by devDept Software: <https://devdept.com/products/eyeshot/>
2. devDept Eshot Code Samples & API: <https://devdept.com/Support/CodeSamples>
3. Model Context Protocol Specification – Anthropic: <https://modelcontextprotocol.io>
4. MCP Python SDK (PyPI): <https://pypi.org/project/mcp/>
5. MCP .NET SDK (GitHub): <https://github.com/modelcontextprotocol/csharp-sdk>
6. Anthropic Tool Use / MCP Documentation: <https://docs.anthropic.com/en/docs/agents-and-tools/mcp>



COMPENDIUM VOL. 2

HACKATHON PROJECT COMPENDIUM

Digital Twins · Energy · Manufacturing · Data Centres · Renewable Energy

Unity · Unreal Engine · NVIDIA Omniverse | Trimiti Innovations Pvt. Ltd., Pune

ID	Project Title	Domain / Focus	Duration
P-09	Smart Pump Health Digital Twin (Component Level)	Digital Twins · Industry 5.0	24–48 hours
P-10	Paint Shop / Coating Line Digital Twin (System-Level Process)	Digital Twins · Manufacturing	24–48 hours
P-11	Solar Plant Performance Digital Twin (Energy Design & Operations)	Digital Twins · Renewable Energy	24–48 hours
P-12	Data Centre Digital Twin — Thermal Risk & Capacity Planning Simulation	Digital Twins · Data Centre	24–48 hours

Smart Pump Health Digital Twin (Component Level)

1. PROBLEM STATEMENT

Industrial pumps consume significant energy and are frequent failure points; unplanned downtime is costly and often preventable with better condition visibility.

2. PROBLEM DESCRIPTION

Centrifugal pumps are among the most common rotating equipment in process and utility industries — yet most operate without any real-time health visibility beyond crude pressure and flow alarms. Bearing wear, cavitation, seal degradation, and dry-run events are the leading failure modes, each with distinct sensor signatures that go undetected in plants relying on periodic manual inspection. Unplanned pump failures trigger cascading production halts, emergency maintenance costs, and safety incidents. The absence of a component-level Digital Twin that correlates multi-sensor data against physics-based performance curves means operators cannot distinguish normal degradation from imminent failure, nor safely explore the impact of off-design operating conditions before attempting them on live equipment.

3. SCOPE & OBJECTIVES

- Visual 3D twin of a centrifugal pump — imported CAD or kitbashed model with annotated subcomponents (impeller, bearings, seals, casing).
- Simulated sensor feeds: suction/discharge pressure, volumetric flow, vibration (RMS/FFT), motor current, and fluid temperature.
- Health Index (0–100%) and Remaining Useful Life (RUL) estimate or failure-risk classification per failure mode.
- Scenario UI to explore off-design operation: throttling effects, cavitation risk onset (NPSHa vs NPSHr), dry-run detection, and viscosity change.

4. DATA & MODELS

- Synthetic time-series datasets covering: normal steady-state operation, bearing wear progression, cavitation onset, seal leak signature, and ambient temperature/viscosity variations.
- Simplified physics-informed model: pump head-vs-flow curve, efficiency-vs-flow curve, and NPSH margin — implemented as look-up tables or polynomial fits.
- Threshold and pattern library mapping sensor deviations to specific failure modes (e.g., elevated vibration at 2× running frequency → impeller imbalance).
- Optional: Isolation Forest or LSTM-Autoencoder for ML-based anomaly detection on top of the rule engine.

5. RECOMMENDED TECHNOLOGY STACK

- 3D / UX: Unity (XR Interaction Toolkit optional) or Unreal Engine 5 (Blueprints) for interactive 3D pump model, alarm overlays, and parameter sliders
- Data Integration: Mock OPC-UA or REST data stream simulated in C# (Unity) or Blueprints/Python microservice; WebSocket bridge for real-time chart updates
- Health Logic: C# or Python service implementing pump curve model, health index formula, and RUL regression or rule-based classifier
- Advanced (Optional): NVIDIA Omniverse (OpenUSD) for higher-fidelity visualization or future integration into a full plant Digital Twin
- Dashboard: Unity UI Toolkit or Unreal UMG for in-scene trend charts, gauges, and scenario control panel

6. CONSTRAINTS & ASSUMPTIONS

- Hackathon duration: 24–48 hours — scope carefully, prototype one complete signal path end-to-end first.
- No full CFD required — simple lumped-parameter or performance-curve-based behaviour is sufficient and expected.
- At least one live signal path (data → model → 3D visualisation) must work end-to-end in the final demo.
- Target hardware: engineering workstation or gaming laptop — avoid GPU-intensive render settings that lock out judges.

7. EXPECTED DELIVERABLES

#	Deliverable
1	Interactive 3D scene: pump model with overlaid gauges, colour-coded health status (green/amber/red), and live trend charts for at least 3 sensor streams.
2	Scenario switcher: demonstrate at least two operating modes (e.g., normal vs. cavitating) with visible KPI impact.
3	Short in-app or slide explanation of the health index formula, assumptions, and failure mode logic.

8. JUDGING CRITERIA

Criterion	What Judges Look For
Model Fidelity vs. Simplicity	Are assumptions clearly stated? Does the twin behave plausibly across normal and fault scenarios?
Actionable Insights	Does an operator know precisely what to do — slow down, schedule maintenance, change setpoint — from the twin's output?
UX & Visualisation Quality	Is the 3D view intuitive under time pressure on a shop floor? Are alarms unambiguous?
Extensibility	How easily can this scale to a multi-pump station or be integrated into an Omniverse plant twin?

9. REFERENCES

1. Unity Digital Twin Use Cases: <https://unity.com/topics/digital-twin-applications-and-use-cases>
2. Unreal Engine for Digital Twins (AnotherReality Studio): <https://anotherreality.studio/digital-twins-unreal-engine/>
3. NVIDIA Omniverse for Sustainable Manufacturing: <https://blogs.nvidia.com/blog/digital-twins-sustainable-manufacturing/>
4. ISO 13374 – Condition monitoring and diagnostics of machines: <https://www.iso.org/standard/78525.html>
5. Scheffer & Girdhar (2004), 'Practical Machinery Vibration Analysis and Predictive Maintenance', Elsevier

Paint Shop / Coating Line Digital Twin (System-Level Process)

1. PROBLEM STATEMENT

Paint shops and coating lines are energy- and capex-intensive, with complex conveyor, oven, and process-window interactions that remain opaque to operators, leading to throughput bottlenecks, energy waste, and quality defects that could be avoided through simulation.

2. PROBLEM DESCRIPTION

Automotive, appliance, and industrial paint shops represent some of the highest energy consumers in a manufacturing facility — industrial ovens alone can account for 40–60% of a paint line's total energy budget. Yet most lines are managed through static SOP setpoints with no mechanism to explore how changes in line speed, oven temperature, or batch mix affect throughput, WIP accumulation, cure quality, and energy cost simultaneously. Discrete-event simulation exists for paint shops in offline tools (AnyLogic, Siemens Plant Simulation), but these are decoupled from the 3D operational view that shift supervisors use to understand live conditions. A Digital Twin that fuses a real-time simulation engine with a 3D process visualisation provides both the analytical depth needed for capacity planning and the intuitive interface needed for operator decisions.

3. SCOPE & OBJECTIVES

- Represent all key process stages: pre-treatment (phosphating/cleaning), spray booths, flash-off zones, curing ovens, cooling, and buffer/accumulation areas.
- Visualise part movement and queueing along a 2D/3D conveyor line with WIP objects and station status indicators.
- Simulate takt time, WIP inventory, bottleneck identification, and basic energy consumption (kWh/unit) per shift.
- Provide operator controls: line speed, oven setpoint temperature, shift pattern (1/2/3 shift), and batch size — with real-time KPI response.

4. DATA & MODELS

- Synthetic station data: cycle times (mean + std dev), equipment failure rates, and rework probabilities per station — loaded from CSV/JSON config files.
- Energy model: kWh vs. oven setpoint and throughput curves for ovens and spray booths — polynomial or look-up table approximation.
- Quality model: mapping of temperature and line speed deviations to defect probabilities (undercure at low temp/high speed, overbake at high temp/low speed, runs/sags at high film build).
- Optional financial overlay: energy tariff (INR/kWh), scrap cost per unit, and OEE calculation to convert operational decisions into rupee impact.

5. RECOMMENDED TECHNOLOGY STACK

- 3D / UX: Unity or Unreal Engine for real-time conveyor visualisation, station status overlays, and WIP object animation along the line
- Simulation Engine: Discrete-event logic in C# (Unity) or Blueprints/Python microservice with WebSocket/REST bridge — support fast-forward and replay for virtual shift simulation
- Data Config: CSV/JSON scenario files for line configuration, demand profile, and station parameters — hot-reloadable without restarting the simulation
- KPI Dashboard: In-engine UI (Unity UI Toolkit / Unreal UMG) showing throughput, OEE proxy, energy/unit, and real-time scrap rate estimate
- Advanced (Optional): NVIDIA Omniverse (OpenUSD) as the high-fidelity factory twin host, with Unity/Unreal as the operator front-end or training interface

6. CONSTRAINTS & ASSUMPTIONS

- Limit to 3–5 station types to keep modelling manageable in hackathon time.
- Simulation must support fast-forward (at least 10× real-time) and replay for a 'virtual shift' demonstration.
- All data sources are synthetic — no real PLC connectivity required, but architecture must clearly show where PLC/BMS data would connect.
- Teams must clearly separate visualisation logic from simulation logic — no monolithic scripts.

7. EXPECTED DELIVERABLES

#	Deliverable
1	Interactive line view showing real-time part status, queue lengths, and station health (idle/running/faulted) in 3D.
2	Scenario editor: ability to change at least 3 operating parameters and observe quantitative KPI changes within the same session.
3	KPI dashboard: throughput (units/shift), OEE proxy, energy consumption (kWh/unit), and estimated scrap rate.

8. JUDGING CRITERIA

Criterion	What Judges Look For
Clarity of Process Representation	Can non-domain judges understand how the paint line works from the twin alone within 2 minutes?
Decision Support Quality	Does the twin make it unambiguous which parameter changes improve throughput or reduce energy — with quantified impact?
Scalability Vision	Is there a clear and credible path to integrate real PLC/BMS data, Omniverse factory models, and external MES/analytics?
Immersion & Usability	Effective use of 3D camera paths, overlays, and alerts for rapid problem diagnosis by a shift supervisor.

9. REFERENCES

1. Unity Industrial Digital Twin eBook: <https://unity.com/resources/industrial-digital-twin-ebook>
2. Unreal Engine for Digital Twins: <https://anotherreality.studio/digital-twins-unreal-engine/>
3. NVIDIA Omniverse for Energy & Manufacturing: <https://www.nvidia.com/en-in/industries/energy/surface-operations/>
4. Mourtzis et al. (2022), 'Digital Twin-based manufacturing system simulation', CIRP Journal, Vol. 112
5. SAP Digital Twins at Work: <https://www.sap.com/blogs/digital-twins-at-work>

Solar Plant Performance Digital Twin (Energy Design & Operations)

1. PROBLEM STATEMENT

Photovoltaic plants routinely underperform against design projections due to progressive degradation, soiling losses, partial shading, and inverter clipping — yet plant owners lack a real-time, actionable Digital Twin that quantifies the root cause and financial impact of each underperformance event.

2. PROBLEM DESCRIPTION

India's solar capacity has surpassed 85 GW and is expanding rapidly under the National Solar Mission and PM Surya Ghar targets, but Performance Ratio (PR) values across rooftop and utility-scale installations frequently fall 5–15% below design assumptions due to unmonitored soiling, string-level mismatch, and inverter degradation. Existing monitoring SCADA tools display raw measurements but do not model expected generation under prevailing conditions, making root-cause isolation slow and empirical. A Digital Twin that fuses a site-specific irradiance-to-power model with live (or synthetic) plant data and a 3D spatial view can pinpoint exactly which strings, panels, or inverters are underperforming and quantify the generation and revenue loss — enabling data-driven O&M decisions such as targeted cleaning schedules and inverter replacement prioritisation.

3. SCOPE & OBJECTIVES

- Model a rooftop or small ground-mounted PV plant: installed capacity (kWp), tilt angle, azimuth, string layout, and inverter configuration.
- Combine weather time-series (GHI/DNI, ambient temperature, wind speed) with a simplified irradiance-to-DC-to-AC power model (including derating factors).
- Compare expected vs. 'measured' (synthetic) plant output and compute KPIs: Performance Ratio (PR), Capacity Utilisation Factor (CUF), specific yield (kWh/kWp).
- Provide interactive scenario tools: soiling level adjustment, string outage simulation, inverter clipping threshold, and partial shading toggle — each with quantified impact on PR and annual revenue loss.

4. DATA & MODELS

- Sample weather time-series: hourly GHI/DNI/DHI and ambient temperature for a representative Indian location (e.g., Pune, Rajasthan) — one year or a representative 2-week period.
- Synthetic 'measured' plant output with injected underperformance events: soiling degradation (gradual), string outage (step drop), sensor offset error, and seasonal dust accumulation profile.
- Panel and inverter datasheet parameters: P_{max}, V_{oc}, I_{sc}, temperature coefficients, inverter efficiency curve — used for DC and AC derating model.
- Optional financial model: applicable tariff (INR/kWh), net metering credit rate, annual degradation rate (0.5–0.7%/yr), and NPV impact of cleaning/maintenance interventions.

5. RECOMMENDED TECHNOLOGY STACK

- 3D / UX: Unity or Unreal Engine to build a geolocated site model (panel arrays + rooftop/terrain) with power flow heatmaps, soiling overlays, and shading visualisation
- Analytics Engine: Python (pvlib library) or C# background service for irradiance → DC → AC power modelling; feeds 3D engine via REST/WebSocket
- Weather Data: NREL NSRDB API or NASA POWER API for real Indian location data; or pre-packaged synthetic CSV for offline hackathon use
- KPI Dashboard: Expected vs. actual yield comparison chart, PR timeline, CUF gauge, anomaly event log — in-engine or web-based React panel
- Advanced (Optional): NVIDIA Omniverse for multi-asset site integration (PV arrays + substation + load centres) or physics-ML shading simulation

6. CONSTRAINTS & ASSUMPTIONS

- Avoid full PVsyst-level simulation complexity — concept-level accuracy with clearly stated assumptions is expected and sufficient.
- Keep simulation time-step coarse (5–15 minutes) to simplify UI rendering and computation within hackathon scope.
- At least one root cause of underperformance must be identifiable and quantifiable in the twin (e.g., soiling reduces PR by X%).
- Must run on a laptop with a mid-range GPU — no cloud-only architectures.

7. EXPECTED DELIVERABLES

#	Deliverable
1	Interactive 3D site view with daily and weekly generation timelines, panel-level colour coding by output, and shading visualisation.
2	KPI panel: expected vs. actual yield, PR, CUF, and automatically identified anomaly events with probable cause label.
3	Scenario sliders/buttons (soiling level, inverter derating, partial shading toggle) with quantitative impact on PR and estimated annual revenue loss (INR).

8. JUDGING CRITERIA

Criterion	What Judges Look For
Insightfulness	How easily can a plant owner see why performance is off and which specific action has the highest ROI?
Use of 3D Spatial View	Does the 3D layout add genuine diagnostic value (e.g., shading patterns, string-level fault isolation) beyond what a 2D chart provides?
Data Rigour	Are model assumptions and validation checks clearly stated — even qualitatively? Is the expected/actual comparison methodology sound?
Platform Integration Vision	Thoughtful use of Unity/Unreal for operator view and Omniverse for enterprise-level multi-site integration — with a credible deployment roadmap.

9. REFERENCES

1. Unity Energy Digital Twin Demo: <https://unity.com/demos/energy-digital-twin>
2. NVIDIA Omniverse for Energy – Surface Operations: <https://www.nvidia.com/en-in/industries/energy/surface-operations/>
3. pvlib Python – PV modelling library: <https://pvlib-python.readthedocs.io>
4. NREL NSRDB Solar Resource Data: <https://nsrdb.nrel.gov>
5. MNRE – National Solar Mission & PM Surya Ghar targets: <https://mnre.gov.in>
6. IEA PVPS Task 13 – Performance and Reliability of Photovoltaic Systems: <https://iea-pvps.org/research-programmes/task-13/>

Data Centre Digital Twin — Thermal Risk & Capacity Planning Simulation

1. PROBLEM STATEMENT

Data centres must balance rising IT power density, cooling capacity, and resilience requirements as AI workloads intensify, yet most operators lack a real-time Digital Twin that simultaneously visualises thermal risk, PUE, and capacity headroom — making capacity planning reactive and prone to costly oversights.

2. PROBLEM DESCRIPTION

Global data centre electricity consumption is projected to double by 2026 driven by AI training and inference workloads. India's hyperscaler and co-location market is expanding at 30%+ CAGR, with new facilities in Mumbai, Chennai, Hyderabad, and Pune. Yet capacity planning — determining when to add cooling, upgrade PDUs, or redistribute rack density — is typically done through static spreadsheets and annual engineering reviews, not through a dynamic, navigable model. Hot aisle/cold aisle containment failures, cooling unit redundancy gaps, and PDU overload conditions are often discovered during live incidents rather than proactively. A Digital Twin that provides a navigable 3D data hall view linked to a real-time thermal and power model enables operators to simulate 'what if I add 20 high-density AI racks?' before procuring hardware, avoiding costly downtime and re-engineering.

3. SCOPE & OBJECTIVES

- Model a single data hall: rack positions, PDU assignments, UPS topology, and CRAC/CRAH cooling unit layout.
- Map IT load profiles to rack inlet temperatures via simplified thermal rules (airflow, containment factor, and supply air temperature).
- Calculate and display PUE in real time, highlight thermal hot spots (rack inlet temp > threshold), and identify overloaded power distribution paths.
- Simulate at least one failure scenario (CRAC outage, UPS failure, PDU trip) and show cascading thermal and power impact across the hall.

4. DATA & MODELS

- Synthetic rack layout: x/y positions in the hall grid, max kW nameplate per rack, current load % — loaded from JSON configuration file.
- Simplified thermal model: temperature rise per rack row as a function of IT load, supply air temperature, and containment effectiveness (hot-aisle containment vs. open floor).
- Electrical model: UPS efficiency curve (% load vs. efficiency), distribution losses per PDU branch, and redundancy topology (N, N+1, 2N).
- IT workload scenario library: baseline steady-state, AI training spike (+40% load on designated GPU racks), batch processing burst, and planned maintenance window (partial load shed).

5. RECOMMENDED TECHNOLOGY STACK

- 3D / UX: Unity or Unreal Engine for a navigable data hall — rack-level temperature colour overlay (blue→green→yellow→red), PDU load bar indicators, and CRAC unit status icons
- Thermal / Power Analytics: Rule-based CFD-lite model in C# (Unity) or Python microservice — temperature estimated per aisle zone using energy balance equations (no full CFD solver required)
- Data Config: JSON rack layout and scenario files — hot-reloadable to allow judges to reconfigure the hall without restarting the twin
- KPI Dashboard: In-engine or web panel showing IT load (kW), facility load (kW), PUE, number of racks exceeding thermal threshold, and UPS reserve capacity (%)

- Advanced (Optional): NVIDIA Omniverse for integrated building-mechanical-electrical twin — Unity/Unreal as the DCIM operator console front-end; connect to real BMS data post-hack

6. CONSTRAINTS & ASSUMPTIONS

- No full CFD solver required — use simple region-based or row-based temperature estimation with clearly documented assumptions.
- Limit the hall to 20–40 racks for performance and visual clarity on typical hardware.
- At least two operating modes (normal load vs. AI training spike) and one failure mode (CRAC outage) must be fully implemented and demonstrable.
- All model parameters (rack density, cooling setpoints, containment type) must be exposed for live tweaking by judges during the demo.

7. EXPECTED DELIVERABLES

#	Deliverable
1	Navigable 3D data hall view with colour-coded racks (thermal risk) and cooling/PDU unit status indicators.
2	KPI dashboard: IT load, facility load, PUE, count of racks above thermal threshold, and UPS reserve capacity.
3	Scenario control panel: adjust rack density, cooling setpoints, containment on/off, and trigger a CRAC/UPS failure test — with immediate visual and KPI response.

8. JUDGING CRITERIA

Criterion	What Judges Look For
Practical Decision Support	Does the twin genuinely help a capacity planner or operations engineer make better decisions — not just visualise data?
Scenario Richness & Clarity	Variety and realism of load and failure scenarios, and clarity of their thermal/power cascading impacts on KPIs.
Integration Vision	Clear and credible explanation of how this twin connects to real BMS/DCIM data sources and an Omniverse-based enterprise twin.
User Experience	Can a judge understand all critical risks within 2–3 minutes of exploration with no briefing?

9. REFERENCES

1. NVIDIA Omniverse for Industrial Digital Twins: <https://blogs.nvidia.com/blog/digital-twins-sustainable-manufacturing/>
2. Unity Industrial Digital Twin eBook: <https://unity.com/resources/industrial-digital-twin-ebook>
3. ASHRAE TC9.9 – Thermal Guidelines for Data Processing Environments (4th Ed.)
4. Uptime Institute PUE Definition and Tiers: <https://uptimeinstitute.com/tiers>
5. IEA Data Centres and Data Transmission Networks – Tracking Report 2024: <https://www.iea.org/energy-system/buildings/data-centres-and-data-transmission-networks>
6. Green Grid PUE White Paper: <https://www.thegreengrid.org/en/resources/library-and-tools/237-pue>

P-13

DEFENCE

Interactive Virtual Walkthrough Trainer for Kalvari-Class Submarines

AR/VR · Digital Twins · 3D Visualisation · Defence · Industry 5.0

Organisation: Indian Navy — Submarine School (SMS), INS Satavahana, Vishakhapatnam □ 24–48 hours

1. PROBLEM STATEMENT

Submariners undergoing basic and conversion training on the Kalvari-class (P-75) submarine at INS Satavahana lack an immersive, interactive training aid — current tools are limited to printed manuals, 2D schematics, and static PowerPoint presentations — resulting in poor spatial familiarisation, restricted procedural practice, and an unnecessarily long learning curve before first access to a live boat.

2. PROBLEM DESCRIPTION

The Submarine School (SMS) at INS Satavahana is the primary training establishment for all Indian submariners. While adequate training aids exist for the older SSK and EKM classes, the Kalvari Basic Course (KBC) and Kalvari Conversion Course (KCC) are conducted almost entirely through printed technical manuals, 2D schematics, and slide-based lectures — the same methods used decades before the Kalvari class entered service. Physical access to the operational Kalvari-class boats (P75 — S21 to S26) is restricted by operational schedules, security protocols, and the finite time each vessel spends alongside. This creates a training capability gap: trainees arrive at a live submarine with limited spatial awareness of compartment layouts, equipment locations, and interdependencies between systems, increasing the cognitive load on experienced crew who must orient them on the boat. An Interactive Virtual Walkthrough Trainer (IVWT) that digitally replicates the Kalvari-class interior and exterior — including IPMS panels, steering console, periscope station, torpedo room, engine room, and all casing fittings — would allow risk-free, repeatable, on-demand training at scale without competing for time on operational assets.

3. SCOPE & OBJECTIVES

- Digitally replicate the Kalvari-class submarine, exterior (bridge fin, casing, casing fittings, masts, appendages) and all interior compartments (control room, torpedo room, engine room, accommodation spaces, battery compartment) at a fidelity level sufficient for spatial familiarisation and procedural training.
- Enable free-roam navigation and guided walkthrough modes through the 3D/VR environment, accessible via standard PC, tablet, and VR headset (multiplatform).
- Implement interactive equipment operation: hatch open/close, deck plate lifting, IPMS panel interaction, steering console, and pressure/temperature gauge feedback — with appropriate visual and audio responses.
- Support foundational submarine evolution training: Dive, Surface, Snorting, Damage Control, and Firefighting procedures as per Indian Navy SOPs.
- Provide trainee analytics: session logs, compartment coverage, procedure step completion, and error events — for instructor review.
- Bonus: AI crew avatar / instructor guide for narrated walkthroughs; AR overlay mode showing equipment nomenclature and system tracing during navigation.

4. DATA & MODELS

- 3D CAD reference data: submarine structural drawings, equipment technical manuals, and piping/electrical layout schematics for the Kalvari class — sourced from Indian Navy / DCNS/Naval Group documentation available at INS Satavahana (data security protocol required; all assets remain on-premise).

- High-resolution photographic reference of compartments, panels, and casing fittings for photorealistic texture authoring (PBR material pipeline).
- IPMS (Integrated Platform Management System) panel layout and indicator logic: lamp states, gauge ranges, alarm conditions — sufficient to simulate operator interaction without exposing classified control software.
- SOPs for key evolutions (Dive, Surface, Snort, DCCF) structured as step-sequence JSON: step ID, compartment, action description, expected system response, go/no-go criteria.
- Trainee analytics schema: session ID, trainee ID, timestamp, compartment visited, interaction event, procedure step, error flag — stored locally with optional export.

5. RECOMMENDED TECHNOLOGY STACK

- 3D Engine: Unity 3D (URP) or Unreal Engine 5 (Lumen disabled on target hardware) — primary development platform; OpenXR for VR headset support (Meta Quest 2/3, Varjo, or equivalent)
- 3D Modelling & Texturing: Blender / Autodesk Maya for geometry authoring; Substance Painter for PBR textures; GLTF/GLB as intermediate interchange format
- Interaction Framework: Unity XR Interaction Toolkit (grab, poke, lever, dial interactions) or Unreal's VR Template — for hatch, panel, and equipment interactions with physics-based feedback
- IPMS / Panel Simulation: Custom C# (Unity) or Blueprint (UE5) state machine mapping panel input events to gauge, lamp, and alarm outputs — no live IPMS software required
- AI / Adaptive Learning (Bonus): Inference via ONNX Runtime (on-device) for trainee performance profiling; AI instructor avatar using Unity Sentis or MetaHuman with text-to-speech narration
- Analytics & Logging: Local SQLite database for session logs; optional REST export to a shore-side analytics dashboard (Python FastAPI + React); all data remains on-premise for security
- Multiplatform Deployment: PC (Windows standalone), Android tablet (Unity IL2CPP build), and VR headset — single codebase with platform-specific quality tiers

6. CONSTRAINTS & ASSUMPTIONS

- All 3D asset creation must use reference material available at INS Satavahana — no classified data may leave the premises; prototype development should plan for a secure, air-gapped build environment if required.
- Classified control software (actual IPMS, weapons systems) is explicitly out of scope — panel simulation is a functional replica, not a live interface.
- Hackathon deliverable: one fully navigable compartment (Control Room) end-to-end with at least three interactive equipment interactions and one SOP procedure (e.g., Diving stations sequence) — full submarine coverage is a post-hackathon roadmap milestone.
- Target hardware: mid-range engineering workstation (NVIDIA RTX 3060 or equivalent) for PC build; Meta Quest 2 for standalone VR — no cloud dependency in the operational prototype.
- Security testing and certification by naval QA agencies is a formal requirement for the production system; the hackathon prototype is a proof-of-concept and not cleared for operational use.
- Assume a 4-person team with Unity/Unreal and 3D modelling skills; photogrammetry of physical submarine components is an acceptable shortcut for rapid geometry if access is granted.

7. EXPECTED DELIVERABLES

#	Deliverable
1	Navigable 3D Control Room prototype: free-roam and guided walkthrough modes, with equipment labels, compartment orientation cues, and at least three interactive elements (hatch, IPMS panel interaction, steering console) with appropriate feedback responses.
2	SOP procedure module: one end-to-end submarine evolution (Diving stations sequence) implemented as a step-guided interactive checklist with go/no-go feedback and a completion score.
3	Multiplatform build: working PC standalone (Windows) and VR headset (Meta Quest 2 or WebXR fallback) builds demonstrated side-by-side during judging.

4	Trainee analytics dashboard: session log with compartments visited, interactions performed, procedure steps completed, and errors flagged — viewable by an instructor on a separate screen.
5	Bonus: AR overlay mode or AI instructor avatar narrating a guided compartment walkthrough with system identification tooltips.

8. JUDGING CRITERIA

Criterion	What Judges Look For
Spatial Fidelity	Does the 3D compartment geometry, equipment placement, and scale accurately reflect the Kalvari-class Control Room as validated by a submarine SME? Gross inaccuracies in layout are a critical defect.
Interaction Realism	Do interactive elements (hatch, panel switches, gauges) respond with physically plausible animations and correct system-state feedback? Does the interaction feel intuitive without controller coaching?
Training Effectiveness	Does the SOP procedure module teach the correct sequence of actions per Indian Navy SOPs? Can a trainee demonstrate improved compartment recall after one walkthrough session?
Multiplatform Performance	Does the VR build maintain ≥ 72 fps on Meta Quest 2 without motion artifacts? Does the PC build run smoothly at 1080p on the target workstation?
Security Architecture	Is the data handling design (on-premise storage, no classified data in cloud, access controls) clearly documented and credible for a naval training environment?
Scalability Roadmap	Is there a credible, costed plan to extend the prototype from one compartment to the full submarine and to adapt the platform for SSN/SSBN classes?

9. REFERENCES

- Unity XR Interaction Toolkit documentation — <https://docs.unity3d.com/Packages/com.unity.xr.interaction.toolkit@3.0/manual/>
- Unreal Engine 5 – VR Template & MetaHuman documentation — <https://docs.unrealengine.com/5.0/en-US/vr-template-in-unreal-engine/>
- Meta Quest Developer Hub – OpenXR SDK — <https://developer.oculus.com/documentation/native/android/mobile-openxr/>
- NVIDIA Omniverse – Digital Twin platform for complex systems — <https://www.nvidia.com/en-us/omniverse/>
- Blender Foundation – Open source 3D creation suite — <https://www.blender.org/>
- Sacks et al. (2013), 'Virtual construction simulation and training', Automation in Construction, Vol. 35
- MIL-HDBK-29612-2A – Instructional Systems Development / Systems Approach to Training and Education (US DoD, applicable as international best practice)
- Indian Navy P-75 Scorpene / Kalvari-class programme overview – Ministry of Defence, Government of India — <https://mod.gov.in>